Izveštaj o jakom i slabom skaliranju

Gausov metod eliminacije - Rust i Python

Darinka Lončar SV31/2022

Cilj ovog rada je analiza performansi paralelne implementacije Gausovog metoda eliminacije kroz eksperimente jakog i slabog skaliranja. Eksperimenti su sprovedeni u Python i Rust implementaciji, a rezultati su upoređeni sa teorijskim granicama definisanim Amdahl-ovim i Gustafson-ovim zakonom.

Tehnički detalji sistema

- Procesor: Apple M4, 10 fizičkih / 10 logičkih jezgara, NUMA:1
- Organizacija: 4 Performance + 6 Efficiency jezgara
- Cache:
  - L1: do 192 KB (instrukcije), 128 KB (podaci)
  - L2: 16 MB (P cluster), 4 MB (E cluster)
- RAM: 16 GB LPDDR5
- OS: macOS 26.3.1
- Python okruženje:
  - Python 3.13.5
  - NumPy 2.4.3
  - Matplotlib 3.10.8
  - multiprocessing (spawn model)
- Rust okruženje:
  - rustic 1.93.1, cargo 1.93.1

Metodologije merenja

Za svaku kombinaciju parametara izvršeno je 30 ponavljanja. Za svaku tačku su izračunati:

- srednje vreme izvršavanja,
- standardna devijacija i

- broj outlier-a.

Outlier-i su pronađeni uz pomoć Tukey parvila: $1.5 \cdot IQR$

Jako skaliranje

Jako skaliranje podrazumeva fiksnu veličinu problema i menjanje broja jezgara, tj. Niti: 1, 2, 4, 8.

Formula ubrzanja: $S_{strong}(p) = \frac{T_{seq}(N)}{T_{par}(N,p)}$, gde je $T_{seq}(N)$ vreme sekvencijalnog izvršavanja problema veličine N, a $T_{par}(N,p)$ vreme paralelnog izvršavanja istog problema na p jezgara.

Slabo skaliranje

Pošto Gausova eliminacija vremenski raste približno sa $O(N^3)$, pri slabom skaliranju dimenzija problema je menjana po pravilu $N(p) = N_0 \cdot p^{1/3}$ kako bi posao po jezgru bio konstantan.

Formula ubrzanja: $S_{weak}(p) = \frac{p \cdot T_{par}(N_0,1)}{T_{par}(N(p),p)}$, gde su nepoznate funkcije prethodno objšnjene.

Paralelizovani deo problema

- Eliminacija redova ispod pivota
  - Za svaki red *i* > *k*: $A[i][j] = A[i][j] - m \cdot A[k][j]$
- Ažuriranje elemenata matrice po redovima
- Računanje faktora eliminacije

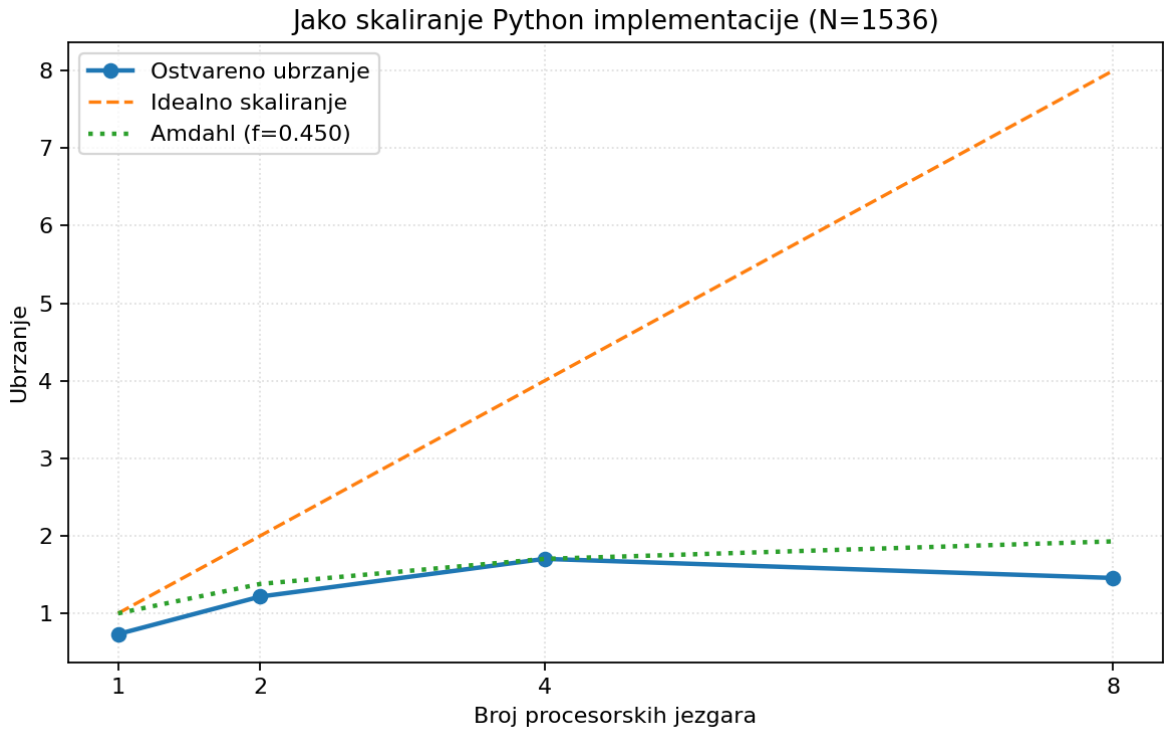
Sekvencijalni deo problema

- Izbor pivota
- Zamena redova
- Iteracije po kolonama
- Supstitucija unazad

Jako skaliranje

Python implementacija

Za jako skaliranje implementacije ovog problema u python-u korišćena je fiksna dimenzija sistema jednačina, tj. matrice N = 1536. Broj niti je menjan (1, 2, 4, 8).



Na narednoj tabeli se nalazi pregled rezultata dobijenih jakim skaliranjem (30 ponavljanja).

Br. Jezgara	Srednje vreme (s)	Std	Outlier-i	Ubrzanje
1	1,859	0,062	5	0,732
2	1.119	0,015	3	1,216
4	0,799	0,045	3	1,702
8	0,935	0,033	3	1,456

Maksimalno ubrzanje za Python jako skaliranje je ~1.702 (na 4 niti), dok je na 8 niti ~1.456. To pokazuje da povećanje broja procesa pomaže samo do određene tačke, posle čega overhead paralelizacije počinje da poništava dobitak.

Amdalov zakon: $S(n) = \frac{1}{(1-p) + \frac{p}{n}}$

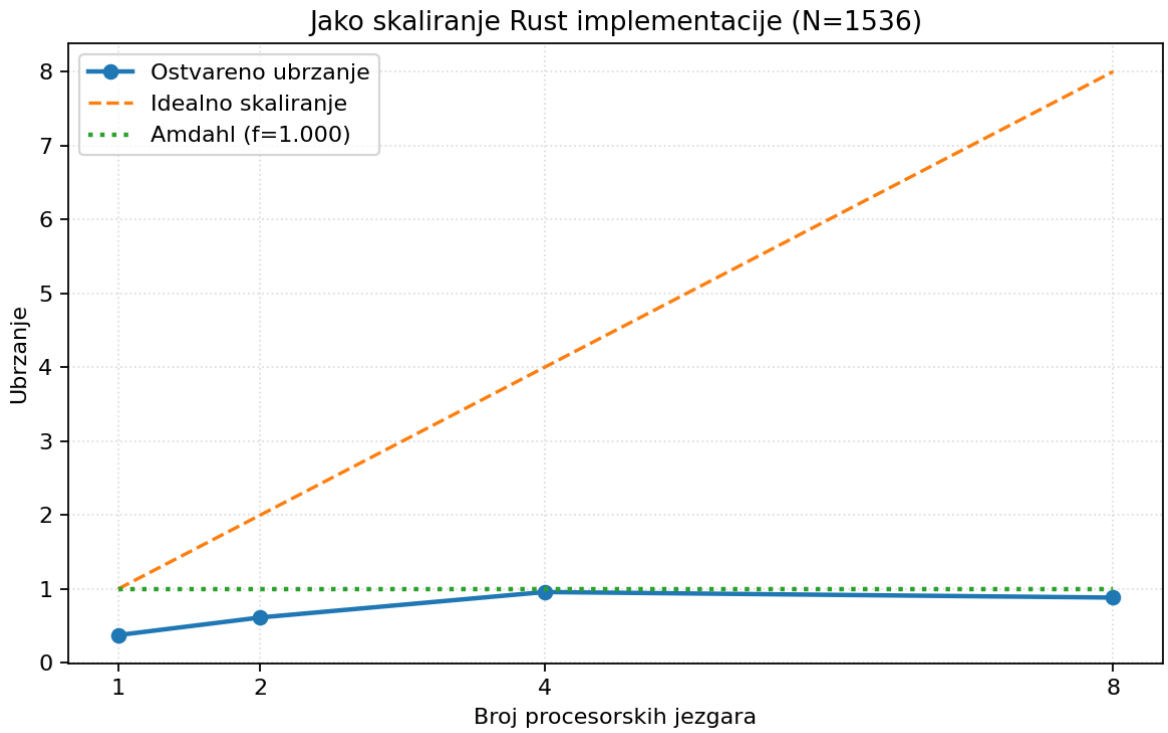
Iz $n = 4, S(4) = 1.702$ => p = 0.55

Dakle paralelizovan deo je oko 55%, a sekvencijalni 45%.

Kada se ovi parametri uvrste u jednačinu $S_{max} = 1/(1-p)$, dobija se teorijsko maksimalno ubrzanje ~ 2.22

Rust implementacija

Za jako skaliranje implementacije ovog problema u Rust-u korišćena je fiksna dimenzija sistema jednačina, tj. Matrice N = 1536. Broj niti je menjan (1, 2, 4, 8).

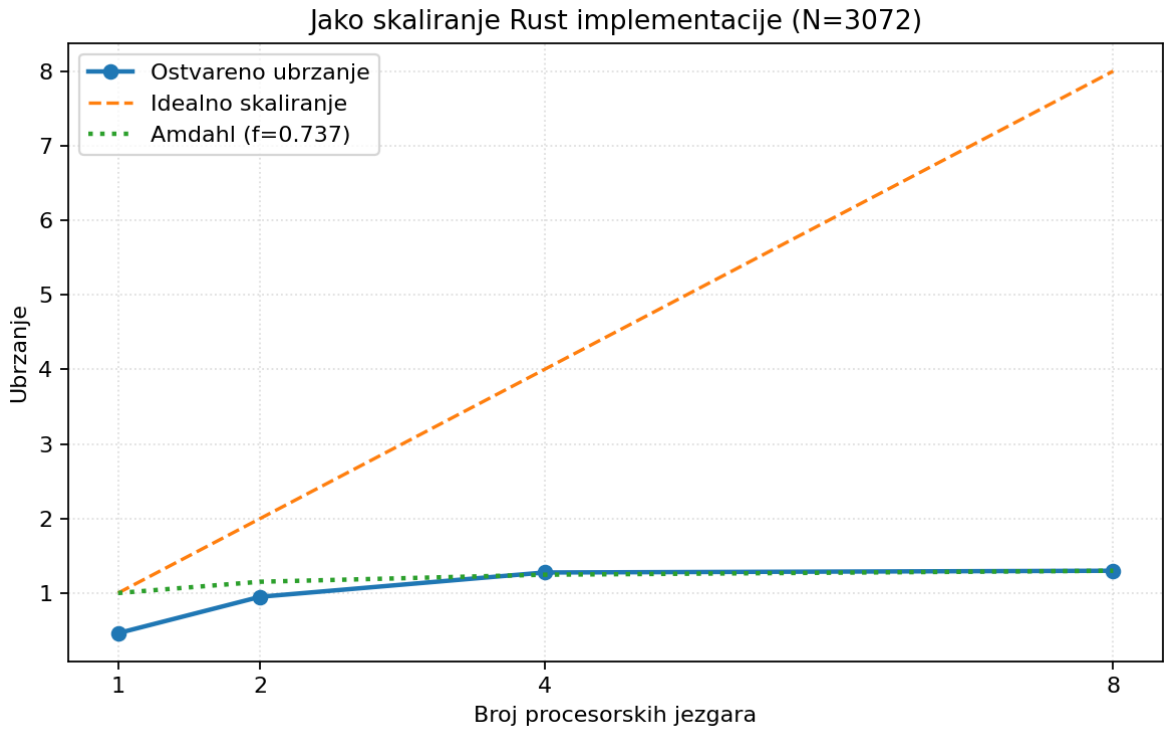


Na narednoj tabeli se nalazi pregled rezultata dobijenih jakim skaliranjem (30 ponavljanja).

Br. Jezgara	Srednje vreme (s)	Std	Outlier-i	Ubrzanje
1	0,533	0,015	4	0,376
2	0,326	0,006	0	0,614
4	0,209	0,018	0	1,958
8	0,227	0,012	0	1,884

Kako nijedna paralelna tačka ne prelazi ubrzanje 1, paralelna implementacija ne donosi performansni dobitak u odnosu na sekvencijalnu.

Za eksperimente sa većom dimenzijom N = 3072 se dobija neki vidi uspešne paralelizacije.



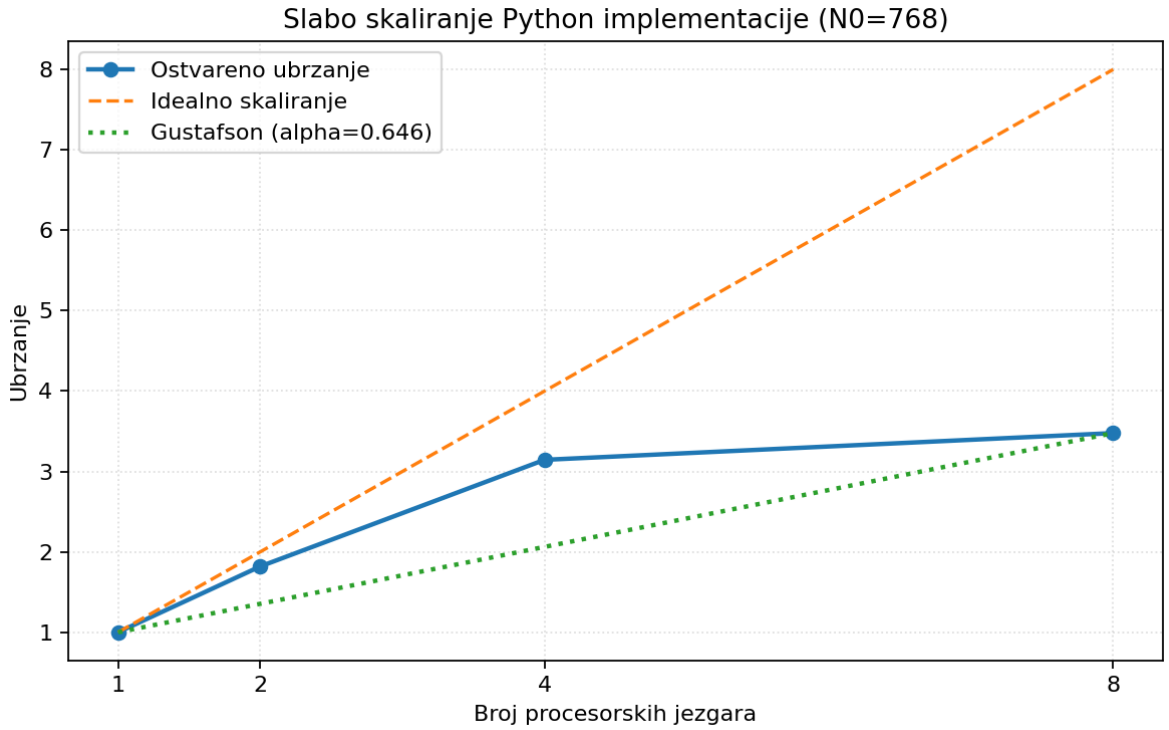
Na narednoj tabeli se nalazi pregled rezultata dobijenih jakim skaliranjem (30 ponavljanja).

Br. Jezgara	Srednje vreme (s)	Std	Outlier-i	Ubrzanje
1	5,048	0,035	3	0,461
2	2,453	0,055	0	0,950
4	1,827	0,034	6	1,275
8	1,794	0,061	2	1,299

Po Amdalu se dobija da je sekvencijalni deo oko 76.35%, a paralelni 23.65%. Iako još uvek preovladava sekvencijalni deo, sada postoji određeni smisao paralelizacije. Dakle, tek sa većim skokom dimenzije problema se vide benefiti paralelizacije u Rust-u.

Slabo skaliranje

Python implementacija



Na narednoj tabeli se nalazi pregled rezultata dobijenih jakim skaliranjem (30 ponavljanja).

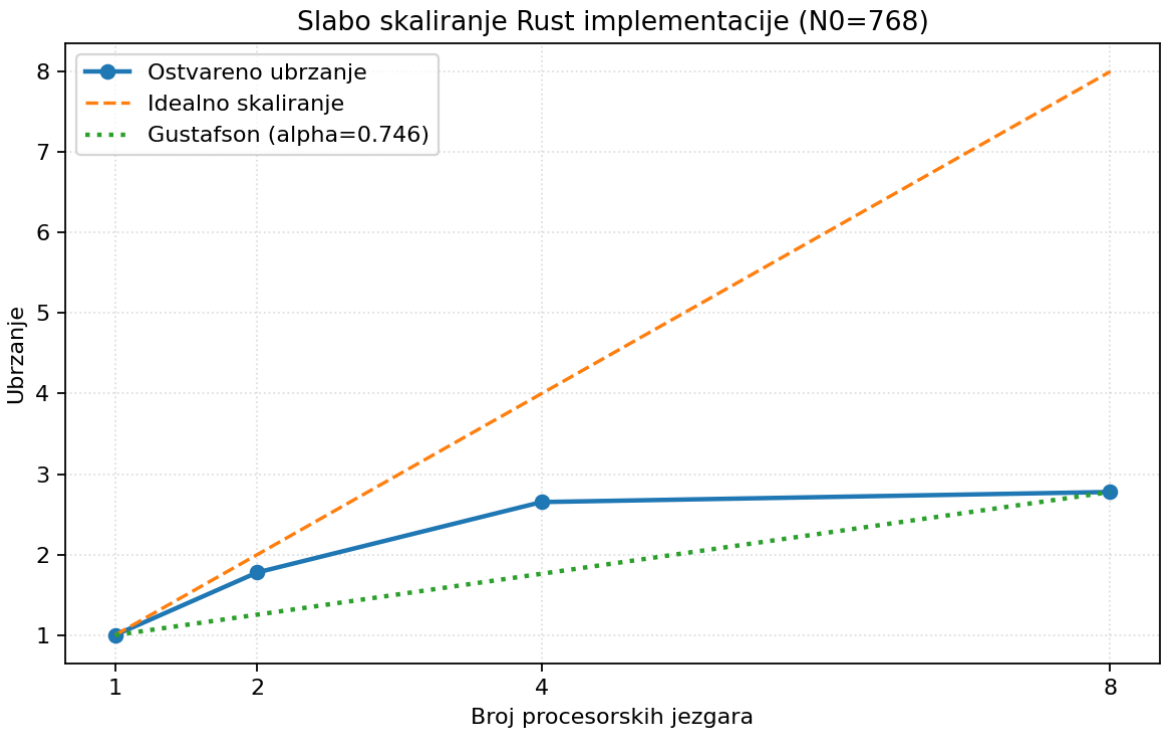
Br. Jezgara	N	Srednje vreme (s)	Std	Outlier-i	Ubrzanje
1	768	0,442	0,015	0	1,000
2	968	0,487	0,007	1	1,819
4	1219	0,563	0,017	8	3,145
8	1536	1,018	0,019	4	3,477

Gustafson zakon: $S(n) = (1 - p) + p \cdot n$

Kada se uvrsti n = 8, $S(n) = 3.447$ => p = 0.3539, tj. paralelni deo iznosi ~ 33.39% i sekvencijalni ~ 66.41%.

Ubrzanje raste do ~3.477 za n = 8 jezgara.

Rust implementacija



Na narednoj tabeli se nalazi pregled rezultata dobijenih jakim skaliranjem (30 ponavljanja).

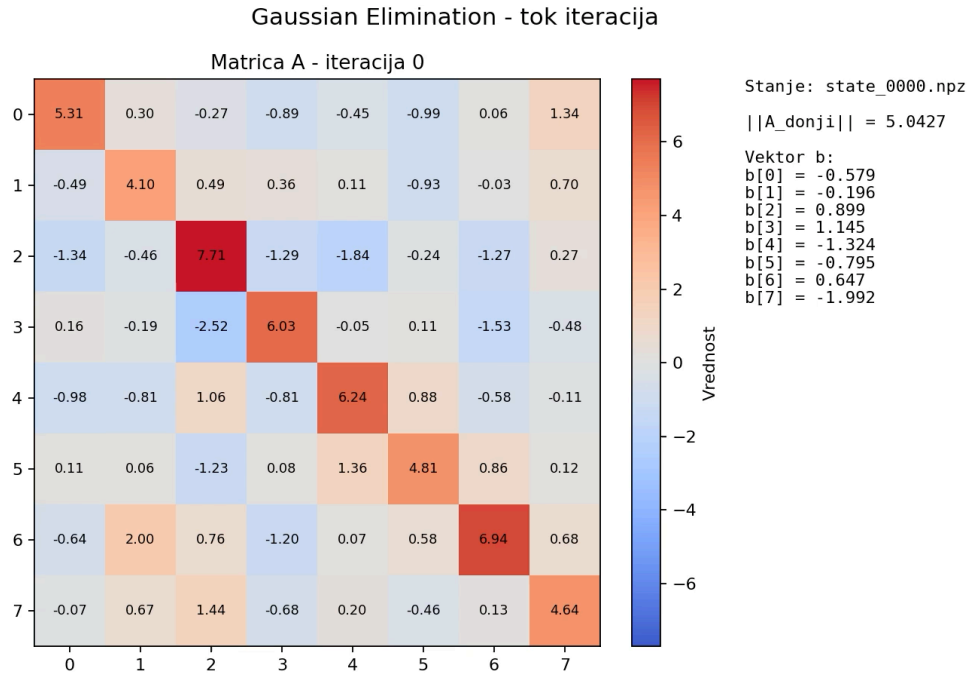
Br. Jezgara	N	Srednje vreme (s)	Std	Outlier-i	Ubrzanje
1	768	0,088	0,005	3	1,000
2	968	0,099	0,007	4	1,778
4	1219	0,132	0,008	3	2,653
8	1536	1,253	0,004	3	2,778

Ponovo se dobija najveće ubrzanje ~ 2.78 za p = 8 jezgara.

Iz Gustafsonovog modela se dobija paralelni deo oko 25.40% i sekvencijalni oko 74.60%, na sličan način kao prethodno izračunavanje.

Vizualizacija problema

Kako bi se dala jasnija slika o problemu, generisan je primer sa manjom matricom N = 8, kako bi se njeni elementi mogli razaznati.



Zaključak

Na osnovu sprovedenih eksperimenata može se zaključiti da efikasnost paralelizacije Gausovog metoda eliminacije značajno zavisi od veličine problema i implementacionog okruženja.

Kod Python implementacije, iako postoji određeni stepen paralelizacije (oko 55% za jako skaliranje), ubrzanje je ograničeno i dostiže maksimum oko 1.7, nakon čega dolazi do pada performansi usled overheada paralelizacije.

Rust implementacija pokazuje drugačije ponašanje. Za manju dimenziju problema (N = 1536), paralelizacija ne donosi značajan dobitak, dok se za veću dimenziju (N = 3072) uočava poboljšanje performansi, što ukazuje na to da je potrebno dovoljno veliko opterećenje kako bi se isplatila paralelizacija.

Rezultati slabog skaliranja pokazuju da Python implementacija ostvaruje umereno povećanje ubrzanja, dok Rust implementacija pokazuje stabilnije ponašanje, iako je sekvencijalni deo i dalje dominantan.

Dakle, ostvareni dobitak paralelizacije u velikoj meri zavisi od implementacije i veličine problema. Overhead, sinhronizacije i upravljanja procesima predstavlja ključni faktor koji ograničava performanse, naročito kod Python implementacije.